

Cấu trúc dữ liệu và thuật toán

Assignment 3

Giảng viên: Nguyễn Thị Tâm

§ Danh sách liên kết §

Phần 1: Mục tiêu

- Nắm được định nghĩa, các thao tác, và độ phức tạp của cấu trúc danh sách liên kết phổ biến gồm danh sách liên kết đơn, danh sách liên kết đôi, danh sách liên kết vòng.
- Đọc hiểu code mẫu và các ví dụ mẫu về danh sách liên kết.
- Làm được các bài tập lý thuyết về danh sách liên kết
- Triển khai được các thuật toán danh sách liên kết trên ngôn ngữ lập trình Java
- Tìm hiểu/sử dụng cấu trúc dữ liệu danh sách liên kết của ngôn ngữ Java

Phần 2: Bài tập thực hành trên lớp

- Tạo danh sách liên kết theo mẫu sau

```
1  // Tao giao dien ListInterface ke thua tu giao dien Iterable
2  public interface ListInterface<T> extends Iterable<T>{
3      public void add(T data);
4      public T get(int i);
5      public int indexOf(T data);
6      public void set(int I, T data);
7      public void remove(int i);
8      public boolean isContain(T data);
9      public int size();
10     public boolean isEmpty();
11 }
12 public class SimpleLinkedList<T> {
13     class Node {
14         T data;
15         Node next;
16     }
17     private Node head = null;
18     int n = 0;
19     @Override
20     public void add(T data) {
21         // TODO Auto-generated method stub
22     }
23     @Override
24     public T get(int i) {
25         // TODO Auto-generated method stub
26     }
27     return null;
28 }
```

```

29     }
30     @Override
31     public void set(int i, T data) {
32         // TODO Auto-generated method stub
33     }
34     }
35     @Override
36     public void remove(int i) {
37         // TODO Auto-generated method stub
38     }
39
40     }
41     @Override
42     public boolean isContain(T data) {
43         // TODO Auto-generated method stub
44
45         return false;
46     }
47     @Override
48     public int indexOf(T data) {
49         // TODO Auto-generated method stubNode p = top;
50
51         return -1;
52     }
53     @Override
54     public int size() {
55         // TODO Auto-generated method stub
56         return 0;
57     }
58     @Override
59     public boolean isEmpty() {
60         // TODO Auto-generated method stub
61         return false;
62     }
63
64     @Override
65     public Iterator<T> iterator() {
66         // TODO Auto-generated method stub
67         return new LinkedListIterator();
68     }
69     class LinkedListIterator implements Iterator<T>{
70
71         @Override
72         public boolean hasNext() {
73             // TODO Auto-generated method stub
74             return false;
75         }
76
77         @Override
78         public T next() {
79             // TODO Auto-generated method stub
80
81             return null;
82         }
83     }
84 }
85
86 }
87
88 }

```

2. Cài đặt danh sách liên kết bằng mảng như sau:

```

1 // Tao lop SimpleArrayList cai dat giao dien ListInterface
2 public class SimpleArrayList<T> implements ListInterface<T> {
3     private T[] array;
4     private int n = 0;
5     private int defaultSize = 100;
6     public SimpleArrayList() {
7         array = (T[]) new Object[defaultSize];
8     }
9     public SimpleArrayList(int capacity) {
10
11     }
12     @Override
13     public void add(T data) {
14     }
15     public T get(int i) {
16         return null;
17     }
18     @Override
19     public void set(int i, T data) {
20     }
21     public void remove(int i) {
22     }
23     @Override
24     public boolean isContain(T data) {
25         return false;
26     }
27     @Override
28     public void remove(int i) {
29         // TODO Auto-generated method stub
30
31     }
32     @Override
33     public int size() {
34         return 0;
35     }
36     @Override
37     public boolean isEmpty() {
38         return false;
39     }
40     public Iterator<T> iterator() {
41         return null;
42     }
43 }
44

```

3. Sử dụng các cấu trúc dữ liệu ở bài 1, 2 để viết chương trình đếm số lần xuất hiện của các từ trong văn bản. Tạo đối tượng là WordCount gồm 2 thuộc tính là word và count. Đối tượng WordCount nạp chồng phương thức equals(Object o) để có thể sử dụng phương thức isContain đã xây dựng ở các cấu trúc dữ liệu trên, hoặc có thể sử dụng phương thức indexOf của các đối tượng cài đặt giao diện List.

Phần 3: Bài tập tự luyện

1. Đảo ngược các phần tử trong danh sách liên kết - <https://practice.geeksforgeeks.org/problems/reverse-a-linked-list/1>
 - Ref - <https://www.geeksforgeeks.org/reverse-a-linked-list/>
 - Tương đương với việc tạo một danh sách liên kết mới thông qua duyệt danh sách liên kết hiện tại và thêm mỗi phần tử của danh sách hiện tại vào đầu danh sách mới.
2. Kiểm tra danh sách liên kết có là palindrome - <https://leetcode.com/problems/palindrome-linked-list/>

- Ref - <https://www.geeksforgeeks.org/function-to-check-if-a-singly-linked-list-is-palindrome/>
 - Tương đương với việc tạo danh sách liên kết đảo ngược và sau đó duyệt đồng thời hai danh sách để kiểm tra hai danh sách có chứa cùng nội dung.
3. Xóa phần tử ở giữa danh sách liên kết - <https://practice.geeksforgeeks.org/problems/delete-middle-of-linked-list/1>
 - Ref: <https://www.geeksforgeeks.org/delete-middle-of-linked-list/>
 4. Trao đổi phần tử thứ k từ đầu danh sách với phần tử thứ k từ cuối danh sách - <https://practice.geeksforgeeks.org/problems/kth-node-from-beginning-and-kth-node-from-end-in-a-singly-linked-list/1>
 - Ref: <https://www.geeksforgeeks.org/swap-kth-node-from-beginning-with-kth-node-from-end-in-a-linked-list/>
 5. Xóa phần tử thứ N từ cuối danh sách - <https://leetcode.com/problems/remove-nth-node-from-end-of-list/>
 - Thực hiện duyệt và xóa trên danh sách liên kết đơn. Xác định vị trí phần tử trước phần tử cần xóa và thực hiện thao tác xóa.
 6. Xóa các phần tử trùng nhau trong danh sách không giảm - <https://leetcode.com/problems/remove-duplicates-from-sorted-list/>
 - So sánh hai phần tử liên kết, nếu có cùng giá trị thì xóa phần tử đứng sau.
 7. Xóa các phần tử có giá trị bằng một giá trị cho trước - <https://leetcode.com/problems/remove-linked-list-elements/>
 - Thực hiện duyệt và xóa trên danh sách liên kết đơn. Cần duy trì con trỏ trỏ đến trước phần tử cần xóa.
 8. Dịch phải danh sách liên kết - <https://leetcode.com/problems/rotate-list/>
 9. Sắp xếp chèn trên danh sách liên kết - <https://leetcode.com/problems/insertion-sort-list/>
 - Mô phỏng Insertion sort trên danh sách liên kết.
 10. Sắp xếp danh sách liên kết - <https://leetcode.com/problems/sort-list/>
 11. Trộn hai danh sách liên kết tăng dần - <https://practice.geeksforgeeks.org/problems/merge-two-sorted-linked-lists/1>
 - Ref: <https://www.geeksforgeeks.org/merge-two-sorted-linked-lists/>
 - Mô phỏng thao tác trộn hai dãy tăng dần và thực hiện trên danh sách liên kết
 12. Trộn xen kẽ hai danh sách liên kết - <https://practice.geeksforgeeks.org/problems/merge-list-alternatingly/1>
 - Ref: <https://www.geeksforgeeks.org/merge-a-linked-list-into-another-linked-list-at-alternate-positions/>
 13. Mergesort trên danh sách liên kết - <https://practice.geeksforgeeks.org/problems/sort-a-linked-list/1>
 - Ref: <https://www.geeksforgeeks.org/merge-sort-for-linked-list/>
 - Thực thi mergesort trên danh sách liên kết.
 14. Quicksort trên danh sách liên kết - <https://www.geeksforgeeks.org/quicksort-on-singly-linked-list/>
 - Ref: <https://practice.geeksforgeeks.org/problems/quick-sort-on-linked-list/1>
 15. Cộng hai đa thức được biểu diễn bằng danh sách liên kết - <https://practice.geeksforgeeks.org/problems/polynomial-addition/1>
 - Ref: <https://www.geeksforgeeks.org/adding-two-polynomials-using-linked-list/>
 16. Nhân hai số được biểu diễn bằng danh sách liên kết - <https://practice.geeksforgeeks.org/problems/multiply-two-linked-lists/1>
 - Ref: <https://www.geeksforgeeks.org/adding-two-polynomials-using-linked-list/>

Phần 4: Tự đánh giá

- ☐ Giải thích được định nghĩa cấu trúc dữ liệu danh sách liên kết và các thao tác trên cấu trúc dữ liệu danh sách liên kết.
- ☐ Có khả năng lựa chọn và triển khai cấu trúc dữ liệu danh sách liên kết đáp ứng nhu cầu bài toán.
- ☐ Có khả năng sử dụng cấu trúc dữ liệu danh sách liên kết trong ngôn ngữ C++.

Tài liệu